

T. P. CHA et K-means

Résumé

Il reprend rapidement des éléments du cours et propose une mise en pratique interactive des algorithmes de clustering. Il reprend rapidement des éléments du cours et propose une mise en pratique de deux approches : la classification ascendante hiérarchique (CAH) avec `hclust()` et la méthode des K-moyenne (k-means) avec `kmeans()`.

1 Fromages

1.1 Chargement des données

Nous traitons d'un ensemble de fromages (29 instances) caractérisés par leurs propriétés nutritionnelles (9 variables).

L'objectif est de déterminer des groupes de fromages homogènes en fonction de leurs propriétés. Nous inspectons et testons deux approches en utilisant deux procédures Python : l'algorithme Hierarchical Agglomerative Clustering (SciPy package) ; et l'algorithme K-Means (scikit-learn package). Le fichier de données "fromage.txt" provient de la page d'enseignement de Marie Chavent de l'Université de Bordeaux.

	calories	sodium	calcium	lipides	retinol	folates	proteines	cholesterol	magnesium
name									
CarreleEst	314	353.5	72.6	26.3	51.6	30.3	21.0	70	20
Babybel	314	238.0	209.8	25.1	63.7	6.4	22.6	70	27
Beaufort	401	112.0	259.4	33.3	54.9	1.2	26.6	120	41
Bleu	342	336.0	211.1	28.9	37.1	27.5	20.2	90	27
Camembert	264	314.0	215.9	19.5	103.0	36.4	23.4	60	20

FIGURE 1 – Premières lignes du dataset Fromage

À vous !

- Chargez les données dans une variable appelée `fromage`.
- Affichez l'écart-type, la valeur min, max et les quartiles des différentes variables.
- Affichez le graph de croisement deux à deux (`pairplot`) de la librairie `seaborn`. Identifiez les variables corrélées.

1.2 CAH

La fonction `linkage` calcule et renvoie des distances deux à deux calculées en utilisant une mesure de distance spécifiée (ou par défaut la distance euclidienne L2) pour calculer les distances entre les individus du dataframe sous formes une matrice de données. La méthode CHA suppose qu'on dispose d'une mesure de dissimilarité entre les individus, dans le cas de points situés dans un espace L2, on peut utiliser la distance comme mesure de dissimilarité. La classification ascendante hiérarchique est dite ascendante car elle part d'une situation où tous les individus sont seuls dans une classe, puis sont rassemblés en classes de plus en plus grandes. Le qualificatif hiérarchique vient du fait qu'elle produit une hiérarchie H à l'ensemble des classes à toutes les étapes de l'algorithme. Dans ce TD nous utilisons la méthode de Ward pour séparer des individus en classes. La méthode de Ward propose qu'à chaque pas, on cherche à obtenir un minimum local de l'inertie intraclasse ou un maximum de l'inertie interclasse. En d'autres termes, elle consiste à réunir les deux clusters dont le regroupement fera le moins baisser l'inertie interclasse. On suppose tout de même l'existence de distances euclidiennes entre observations. Cette technique tend à regrouper les petites classes entre elles.

```
> from matplotlib import pyplot as plt
> from scipy.cluster.hierarchy import dendrogram, linkage
> from pylab import rcParams
> rcParams['figure.figsize'] = 8, 10
#generate the linkage matrix      Ward method
> Z = linkage(fromage,method='ward',metric='euclidean')
#plotting the dendrogram
> plt.title("CAH")
> dendrogram(Z,labels=fromage.index,orientation='left',
             color_threshold=0)
> plt.show()
```

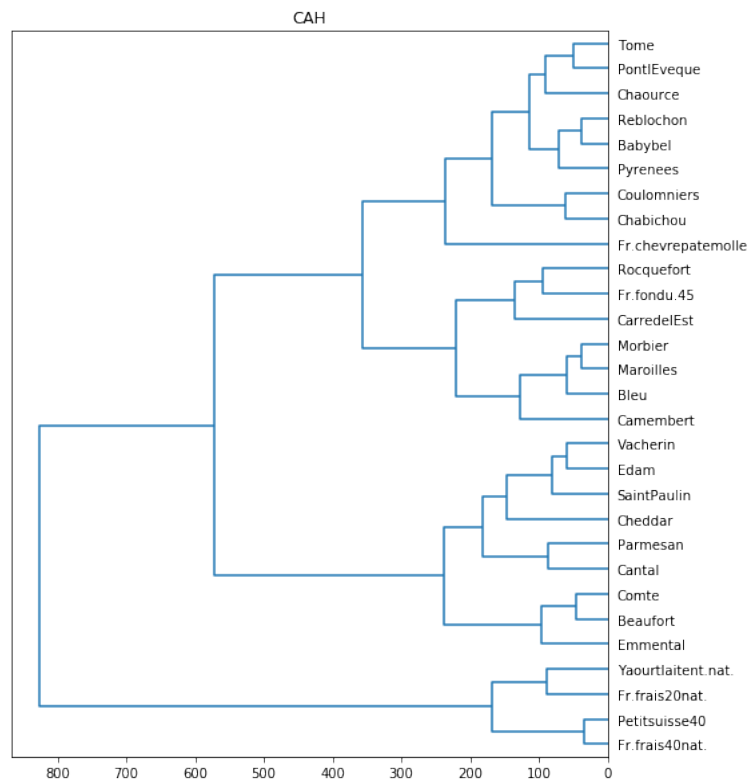


FIGURE 3 – Résultat du CHA (méthode Ward, Euclidienne) sur le dataset Fromage

À vous !

- Quel découpage vous semble optimale (justifiez) ?
- On décide de faire un découpage en 4 groupes. Identifiez et décrivez les caractéristiques des quatres groupes.
- Regardez les paramètre de la méthode et essayez un autre paramétrage (de votre choix). Comparez avec notre modèle.

2 Iris

2.1 Chargement des données



FIGURE 4 – Caractéristique des iris

Le jeu de données Iris a été initialement publié à l'UCI Machine Learning Repository : Iris Data Set. Ce Data Set de 1936 est souvent utilisé pour tester des algorithmes d'apprentissage automatique et des visualisations. Le jeu de données Iris contient trois variantes de la fleur Iris. Il contient 150 instances (ligne du jeu de données). Chaque instance est composée de quatre attributs pour décrire une fleur d'Iris. Le jeu de données est étiquetée par le type de fleur. Ainsi pour quatre attributs décrivant une fleur d'Iris, on saura de quelle variante il s'agit. Le jeu de données ne contient pas de valeurs manquantes. Ce qui nous dispense de les traiter.

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
Id					
1	5.1		3.5	1.4	0.2 Iris-setosa
2	4.9		3.0	1.4	0.2 Iris-setosa
3	4.7		3.2	1.3	0.2 Iris-setosa
4	4.6		3.1	1.5	0.2 Iris-setosa
5	5.0		3.6	1.4	0.2 Iris-setosa

FIGURE 5 – Premières lignes du dataset Iris

À vous !

- Chargez les données dans une variable appelée `iris`.
- Affichez l'écart-type, la valeur min, max et les quartiles des différentes variables.
- Affichez le graph de croisement deux à deux (`pairplot`) de la librairie `seaborn`. Identifiez les variables corrélées.

2.2 K-mean

La méthode dite *K-means* propose de choisir aléatoirement un point comme le barycentre de chaque groupe puis de le recalculer à chaque nouvel individu introduit dans le groupe. Cette technique est intéressante car elle s'adapte lorsqu'on injecte de nouveaux individus. Comme nous souhaitons avoir des résultats reproductible entre nous, nous allons 'forcer' l'aléatoire à l'aide de la fonction `random_seed`. *K-means* demande également de fixer un hyperparamètre qui est le nombre de clusters. Ce nombre est inconnu *A priori* et c'est pourquoi *K-means* est une méthode dite 'non supervisée'. Dans notre exemple, nous connaissons le nombre d'espèces (l'objectif ici étant de vous convaincre que la méthode marche), mais en pratique, nous n'avez pas cette information. Commençons par faire tourner *K-means* avec $k = 2$.

```
> from sklearn.cluster import KMeans
> import random

> random.seed(10)

#Cluster K-means
> model=KMeans(n_clusters=2,random_state = 0)
```

```
#adapter le modèle de données
> model.fit(iris[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm',
                  'PetalWidthCm']])
> res_label = model.labels_
> res_label
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=
      =int32)
```

Le code précédent nous dit quelle appartenance de chaque fleur à quelle cluster. Toutefois, un tableau de la sorte n'est pas très parlant. Vu que notre jeu de données est relativement petit, on peut visualiser graphiquement notre jeu de données pour observer les clusters formés. En nous basant sur la longueur et la largeur de chaque pétale, on peut afficher, dans un plan 2D, les différentes fleurs de notre jeu de données. Visuellement, on voit qu'il y a deux grands groupes qui se forment.

```
#On ajoute une colonne qui contiendra le cluster choisi par kmeans
> iris.insert(5, "kmeansClass", model.labels_, True)
> iris.head()
      SepalLengthCm  SepalWidthCm  PetalLengthCm
      PetalWidthCm  Species
Id
1      5.1      3.5      1.4      0.2  Iris-setosa  0
2      4.9      3.0      1.4      0.2  Iris-setosa  0
3      4.7      3.2      1.3      0.2  Iris-setosa  0
4      4.6      3.1      1.5      0.2  Iris-setosa  0
5      5.0      3.6      1.4      0.2  Iris-setosa  0

> import seaborn as sns
> sns.lmplot('SepalLengthCm', 'SepalWidthCm', data=iris, hue='
      kmeansClass', fit_reg=False)
> plt.show()
```

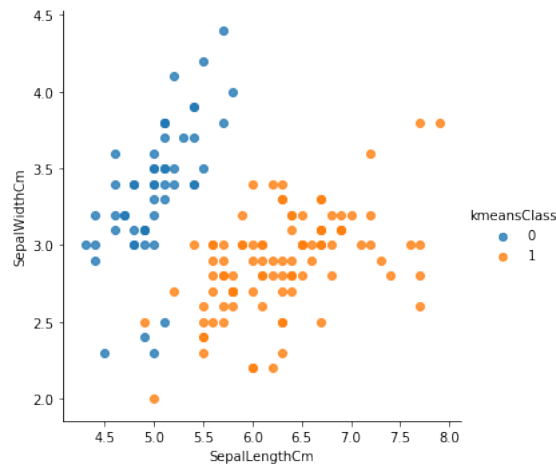


FIGURE 6 – Scatter 'SepalLengthCm', 'SepalWidthCm' selon les classes de kmeans sur Iris

À vous !

- K-means* peut-il utiliser des variables catégorielles pour construire les groupes ? Pourquoi ?
- Affichez à l'aide de la fonction `lplot` et variable 'SepalLengthCm' en fonction de 'SepalWidthCm', avec comme couleur de point la variable 'Species'. Comparez avec le graphe 2.3. Combien d'espèces y-a-t-il vraiment d'espèce dans ce jeu de données ?
- Réalisez un k-means avec le nombre de classe égale au vrais nombre d'espèce.
- Comparer ce clustering proposé par rapport aux vrais espèces. Sur quels espèces k-means est il en difficulté ? Pour cela utilisez la fonction `crosstab`.

2.3 Nombre de classe

K-means, à la différence de la CAH, ne fournit pas d'outil d'aide à la détection du nombre de classes. Bien qu'ici vous ayez accès au vrais nombre de classes différentes, cette information n'est pas supposée être connue dans le cas de l'apprentissage non supervisé. Nous allons maintenant faire comme si nous ne connaissions pas cette information. Nous devons les programmer sous python ou utiliser des procédures proposées par des packages dédiés. En faisant varier le nombre de groupes on observe l'évolution d'un indicateur sur l'aptitude des individus à être plus proches entre eux dans un même groupe, que des individus des autres groupes. Cet indicateur traduit de la qualité d'une solution. On propose ici d'observer l'évolution de la somme des carrés au sein d'un cluster (Within Cluster Sum of Squares, WCSS), qui mesure la distance moyenne au carré de tous les points d'un cluster par rapport à son centroïde, et ce pour différentes partitions. On cherche ensuite le point où la tangente devient horizontale dans le graphique (plus d'évolution). Cette méthode est aussi appelée 'la règle du coude' ou 'Elbow method' en anglais.

```

# to store WCSS
> wcss = []

# for loop
> for i in range(1, 11):

    # k-mean cluster model for different k values
    kmeans=KMeans(n_clusters=i,random_state = 0)
    kmeans.fit(iris[['SepalLengthCm', 'SepalWidthCm', '
        PetalLengthCm', 'PetalWidthCm']])

    # inertia method returns wcss for that model
    wcss.append(kmeans.inertia_)

# figure size
> plt.figure(figsize=(10,5))
> sns.lineplot(range(1, 11), wcss,marker='o',color='green')

# labeling
> plt.title('The Elbow Method')
> plt.xlabel('Number of clusters')
> plt.ylabel('WCSS')
> plt.show()

```

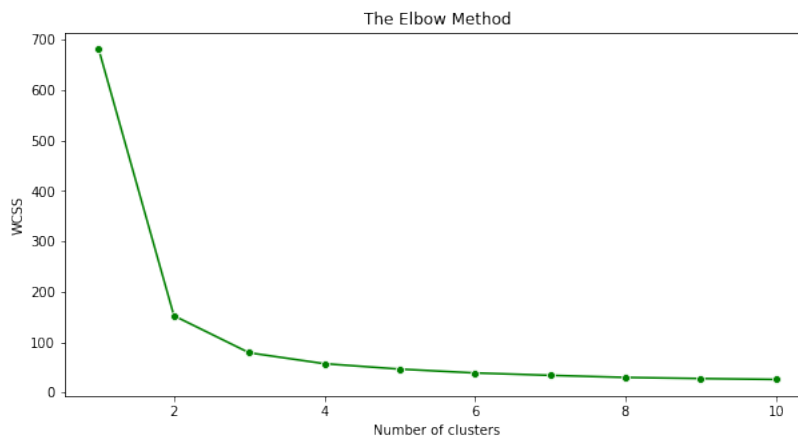


FIGURE 7 – Méthode du coude pour l'évaluation du nombre de groupe de kmeans sur Iris

À vous !

- Conclure sur le nombre k à fixer.
- Réalisez une C.H.A sur le dataset Iris et comparer avec k-means à l'aide d'une matrice de confusion.
- Conclure sur les différences entre les deux méthodes de clustering pour le dataset Iris¹.

1. Pour obtenir les données labélisées avec une CHA vous pouvez utiliser la fonction `fcluster`

3 Pour aller plus loin

<https://hands-on.cloud/implementation-of-k-means-clustering-algorithm-using-python>.